

Schildy – Android App



Software-Engineering
Media Systems SoSe 2024
HAW Hamburg

Prof. Dr. Larissa Putzar

Name (Mat. Nr)
Name (Mat. Nr)
Name (Mat. Nr)

Inhaltsverzeichnis

1 EINLEITUNG	1
2 LASTENHEFT	1
2.1 ZIELBESTIMMUNG.....	1
2.2 PRODUKTEINSATZ.....	1
2.3 PRODUKTFUNKTIONEN.....	1
2.4 PRODUKTDATEN.....	1
2.5 PRODUKTLEISTUNGEN.....	2
2.6 QUALITÄTSANFORDERUNGEN.....	2
2.7 ERGÄNZUNGEN.....	2
2.8 SICHERHEITSANFORDERUNGEN.....	3
2.9 FUNKTIONSKLASSIFIKATION.....	3
3 PFLICHTENHEFT	4
3.1 ECKDATEN.....	4
3.1.1 Verantwortliche	4
3.1.2 Version	4
3.1.3 Änderungshistorie	4
3.2 EINLEITUNG.....	4
3.2.1 Motivation	4
3.2.2 Vision	4
3.2.3 Zielgruppe	4
3.3 RANDBEDINGUNGEN.....	5
3.3.1 Entwicklungsumgebung	5
3.3.2 Werkzeuge	5
3.3.3 Sprachen	5
3.4 SPEZIFIKATION DER ANFORDERUNG.....	5
3.4.1 Funktionale Anforderungen	5
3.4.2 Nicht-Funktionale Anforderungen	8
4 DARSTELLUNG DES ENTWURFS DER SOFTWARE-ARCHITEKTUR	9
4.1 SOFTWAREPATTERN.....	9
4.2 ENTWURFSPRINZIPIEN.....	9
4.2.1 DRY – Don't Repeat Yourself	9
4.2.2 SEPARATION OF CONCERNS.....	10
4.2.3 UML-DIAGRAMME.....	10
4.2.1 Use-Case-Diagramm:	10
4.2.2 Component Diagramm:	10
4.2.3 Sequenzdiagramm:	11
4.2.4 Klassendiagramme	12
5 DARSTELLUNG DER IMPLEMENTATION	14
6 TEST	14
7 CI/CD	14
8. VORGEHENSMODELL UND ZUSAMMENARBEIT	15
8.2 ZUSAMMENARBEIT TEAM.....	15
8.2.1 Aufwandseinschätzung:	16
9 LESSONS LEARNED	16
10 LINKS.....	16

1 Einleitung

Im folgenden Projektbericht des Software-Engineering Projektes "Schildy – Android App" von Team 8 wird erläutert, wie diese App zustande gekommen ist, mithilfe der Idee sowie Zielbestimmung, Anforderungen und Darstellung der Software-Architektur und Implementation. Es wird außerdem die Teamarbeit sowie das Vorgehensmodell erläutert. Abschließend werden die Lessons Learned aus diesem Projekt erläutert.

2 Lastenheft

Das Lastenheft dient dazu die Anforderungen an das entstehende Produkt zu formulieren. Dies wird im Folgenden ausgeführt.

2.1 Zielbestimmung

Die App "Schildy" soll es Verkehrsteilnehmer ermöglichen, ohne komplizierte Beschreibung Verkehrsschilder zu finden, um deren Bedeutung nachzuschauen. Die Suche soll auf einer Eigenschafts-basierten Filterfunktion basieren. Zudem soll es eine Favoritenliste geben. (Nice to have: Trainingsfunktion)

2.2 Produkteinsatz

Das Produkt soll auf Android Smartphones laufen und von Verkehrsanfänger, unsicheren Fahrer oder auch Beifahrer genutzt werden.

2.3 Produktfunktionen

- /LF10/ („Schnickschnack“) Übungsfunktion mit den Schildern in den Favoriten
- /LF20/ Erfassung, Änderung und Löschung von Verkehrsschildern (Admin).
- /LF30/ Detailansicht für weitere Informationen über das ausgewählte Schild
- /LF40/ Funktion, Favoriten zu speichern und separat anzeigen zu lassen
- /LF50/ Erfassung, Änderung und Löschung der Nutzeraccounts.
- /LF60/ Sortieren nach Eigenschaften und mithilfe einer Suchfunktion.
- /LF70/ Fehlerfreie Ausgabe der erfassten Verkehrsschilder.

2.4 Produktdaten

- /LD10/ Folgende Daten sind zu jedem Verkehrsschild zu speichern:
Name, Form, Farbe, Beschreibung, Bild
- /LD20/ Accounts benötigen folgende Daten: Favoritenliste, Name, E-Mail, Passwort.

2.5 Produktleistungen

- /LL10/ Bei der Listenausgabe der Funktionen /LF50/ werden zunächst alle Schilder ausgeben, Einschränkungen mithilfe der Filter möglich.
- /LL20/ Die Filterung der Suche sollte unter 0.5 Sekunden dauern.
- /LL30/ Die Funktionen /LF40/ und /LF50/ müssen jedem Nutzer zur Verfügung stehen, um seine App zu personalisieren.
- /LL40/ Die Verkehrsschilder, die ein Admin hinzufügt, müssen Eingabetreu übernommen und überprüft werden. Keine doppelten Eingaben erlaubt.
- /LL50/ Alle offiziell zugelassen Verkehrsschilder speichern können.
- /LL60/ Produkt sollte auf den meisten Android Smartphones laufen.
- /LL70/ Verwendung des Produkts soll intuitiv und effizient sein.

2.6 Qualitätsanforderungen

Produktqualität	Sehr gut	gut	normal	irrelevant
Funktionalität		x		
Zuverlässigkeit	x			
Benutzbarkeit	x			
Effizienz		x	x	
Änderbarkeit		x		
Portierbarkeit				x

2.7 Ergänzungen

- Sicherheitsanforderungen Accountverwaltung / Login
- Geschätzter Aufwand:
50-60h pro Person
- Risiko:
Größtes Risiko ist der Umgang mit einer neuen Umgebung (Android Studio), indem keine Erfahrungen vorhanden sind und wodurch eine Einarbeitung mit in den Zeitaufwand mit einberechnet werden muss.

- **Stabilität:**
Sehr hohe Wahrscheinlichkeit, da die Umsetzung des Lastenheftes aufgrund mangelnder Erfahrung nicht eindeutig festgelegt werden konnte. Da hiermit gerechnet wird, haben wir den Ablaufplan entsprechend gestaltet.
- „Schnickschnack“:
Eine Übungsfunktion, mit der man die Eigenschaften der Favoriten durch ein einfaches Abfrageprinzip üben kann

2.8 Sicherheitsanforderungen

- Passwort darf nicht in Klartext gespeichert werden.
 - Nutzerdaten dürfen nur an den bestimmten Nutzer weitergegeben werden
- Weitere Sicherheitsanforderungen sind nicht notwendig, da die Daten nicht sensibel sind.

2.9 Funktionsklassifikation

- **Accounts: Erstellen, Login, Bearbeiten**
 - Aufwand: Hoch
 - Risiko: Hoch
 - Stabilität: Niedrig
 - Grund: Ermöglicht die Nutzung der App auf einer Individuellen Ebene
- **Schilder: Erstellen, Bearbeiten, Löschen (Admin)**
 - Aufwand: Hoch
 - Risiko: Mittels
 - Stabilität: Mittel
 - Grund: Änderung/Erweiterung der Datenbank über ein User Interface / Admin-Account sehr hilfreich
- **Filter: Filtern nach Eigenschaften oder per Suchleiste**
 - Aufwand: Hoch
 - Risiko: Niedrig
 - Stabilität: Niedrig
 - Grund: Wichtigste Eigenschaft für die Nutzbarkeit und Daseinsberechtigung der App

3 Pflichtenheft

Im Folgenden Abschnitt wird das Pflichtenheft der Applikation, welches anhand der im Lastenheft vorhandenen Anforderungen erstellt wurde, dargestellt.

3.1 Eckdaten

3.1.1 Verantwortliche

Name

Name

Name

3.1.2 Version

Version der App: 1.0

3.1.3 Änderungshistorie

Die App wurde während der Entwicklung trotz des recht statischen „V-Modell“, laufend angepasst, da aufgrund fehlender Erfahrung einige Anpassungen nötig waren. Daher sind Funktionen, die vorher jedoch auch als „Schnickschnack“ definiert wurden, ausgelassen wurden und haben es aus zeitlichen Gründen nicht mehr in die App geschafft.

3.2 Einleitung

3.2.1 Motivation

Die Motivation diese App zu gestalten war, dass im Straßenverkehr die Situation vorkommen kann, in der Verkehrsteilnehmer sich nicht sicher über die Bedeutung eines Verkehrsschildes sind. Das Ziel war es, dass sowohl Fahranfänger, unsichere Verkehrsteilnehmer und auch Beifahrer schnell die Bedeutung der im Straßenverkehr gesichteten Schilder herauszufinden.

3.2.2 Vision

Die Vision ist es, den Straßenverkehr sicherer zu machen, indem das Wissen über die Verkehrsschilder und deren Bedeutung vertieft wird und zur Not schnell und einfach zur Verfügung steht.

3.2.3 Zielgruppe

Die Zielgruppe sind Fahranfänger, unsichere Verkehrsteilnehmer sowie Beifahrer, welche die Möglichkeit bekommen, Bedeutung von Verkehrsschildern effizient nachzuschlagen.

3.3 Randbedingungen

3.3.1 Entwicklungsumgebung

Die Entwicklungsumgebung für die sich entschieden wurde ist Android Studio.

3.3.2 Werkzeuge

Verwendete Werkzeuge beinhalten Android Studio und enthaltene Packages als Entwicklungsumgebung, GitHub für die Versionsverwaltung, SQLite 3 für die Realisierung der Datenbankimplementation, Miro Board¹ als Projektplaner und Verwalter, ChatGPT, vor allem für das Erstellen initialer Unittests sowie Draw.io für die Erstellung der benötigten Diagramme.

3.3.3 Sprachen

Die verwendete Programmiersprache ist Java für den Hauptteil der Applikation sowie die Auszeichnungssprache XML für die Darstellung der Daten als „Frontend“ der App. Des Weiteren wurde SQL verwendet, um Daten mittels der Datenbank abzufragen und einzufügen.

3.4 Spezifikation der Anforderung

Die Anforderungen aus dem Lastenheft ergeben im Folgenden sowohl was die App können muss als auch wie sie dies realisiert und welche nicht-funktionalen Anforderungen umgesetzt werden müssen.

3.4.1 Funktionale Anforderungen

/PF10/ (Wurde nicht umgesetzt)

Name: Trainingsfunktion

Akteur: Nutzer

Ereignisse: Es wird dem Nutzer eine Trainingsfunktion der Verkehrsschilder angeboten

Kurzbeschreibung: Die App erlaubt dem Nutzer eine Trainingsfunktion zu nutzen, mit der der Wissenstand über die Verkehrsschilder und ihrer Bedeutungen überprüft sowie verbessert werden kann.

Vorbedingungen: Der Nutzer muss eingeloggt sein

Ablaufschritte:

1. Der Nutzer authentifiziert sich über den Login
2. Der Nutzer ruft in dem Navigationsmenü den Reiter „Training“ auf

¹ siehe 10 Links

3. Der Nutzer bekommt Fragen über die Verkehrsschilder gestellt und beantwortet diese.
4. Am Ende bekommt der Nutzer ein Ergebnis seiner Leistung.

/PF20/

Name: Verwaltung der Verkehrsschilder

Akteur: Admin

Ereignisse: Verkehrsschilder werden angepasst

Kurzbeschreibung: Die Admins sollen Zugriff auf die Verkehrsschilder haben und diese hinzufügen, ändern und löschen können.

Vorbedingungen: Der Nutzer muss als Admin authentifiziert, sein

Ablaufschritte:

1. Der Admin authentifiziert sich über den Login
2. Der Admin navigiert im Navigationsmenü auf die Bearbeitungsseite der Verkehrsschilder
3. Der Admin gibt in die vorgegeben Felder die Daten /LD10/ des neuen Verkehrsschildes ein
4. Der Admin klickt auf den „Add“-Button und navigiert zurück auf die „Home“-Seite

/PF30

Name: Detailansicht

Akteur: Nutzer

Ereignisse: Nach dem Klicken auf ein Schild wird eine Detailansicht geöffnet.

Kurzbeschreibung: Der Nutzer soll eine Detailansicht der Verkehrsschilder bereitgestellt bekommen, in der die Bezeichnung, Bild und die Beschreibung beziehungsweise Bedeutung des Verkehrsschildes vorliegt.

Vorbedingungen: Die Verkehrsschilderliste wird mit angewendeten Filtern ausgegeben.

/PF40/

Name: Favoritenliste

Akteur: Nutzer*in

Ereignisse: Nutzer können Verkehrsschilder als Favoriten markieren; diese werden in einer personalisierten Favoritenliste angezeigt.

Kurzbeschreibung: In der Detailansicht der Verkehrsschilder wird ein Favoritensymbol angezeigt, wo der Nutzer draufklicken kann. Darauf wird dieses Verkehrsschild in die Favoritenliste des Nutzers hinzugefügt, welche er sich in einem Navigationsmenü anzeigen lassen kann.

Vorbedingungen: Dem Nutzer muss eingeloggt sein und Favoriten hinzugefügt haben.

/PF50/

Name: Nutzerverwaltung

Akteure: Nutzer, Admin

Ereignisse: Nutzeraccounts werden erstellt, bearbeitet und gelöscht

Kurzbeschreibung: Nutzer und Admins können sich mit ihren Accounts anmelden und diese bearbeiten. Es gibt spezielle Funktionen wie beispielsweise eine Favoritenliste und die Bearbeitung der Verkehrsschilder für Admins.

Vorbedingungen: Es muss eine Navigationsseite vorhanden sein auf welcher sich Nutzer*in und Admins registrieren/einloggen können.

/PF60/

Name: Filterfunktion nach Eigenschaften der Verkehrsschilder

Akteure: Nutzer

Ereignisse: Nutzer*in klicken Filterbuttons an und erhalten eine gefilterte Ausgabe der Schilder

Kurzbeschreibung: Es wird die Möglichkeit zur Verfügung gestellt, bei der Suchfunktion zusätzlich Buttons anzuklicken, welche dafür sorgen, dass die Nutzer eine gefilterte Ausgabe der Verkehrsschilder, basierend auf deren Eigenschaften wie Form, Farbe oder Bezeichnung, erhält und beliebig anpassen kann.

Vorbedingungen: Es muss ein Filtermenü mit allen möglichen Optionen existieren

/PF70/

Name: Ausgabe der Verkehrsschilder

Akteure: Nutzer

Ereignisse: Nutzer betätigt Filterbuttons, nutzt die Suchfunktion oder scrollt durch die Verkehrsschilder, um sich diese anzuschauen.

Kurzbeschreibung: Nutzer erhält verschieden Möglichkeiten, Anpassungen an der Suche vorzunehmen wie etwa das Anklicken der Filterbuttons oder das Nutzen der Suchleiste, wobei die den Optionen entsprechend und richtig ausgegeben werden.

Vorbedingung: Die Eigenschaften der Verkehrsschilder müssen stimmen und die Filterfunktionen richtig funktionieren

3.4.2 Nicht-Funktionale Anforderungen

/PL10/

Name: Einstellung der Suchergebnisse

Akteure: Nutzer

Kurzbeschreibung: Die Ausgabe der Verkehrsschilder soll zunächst alle beinhalten und nicht geändert werden, solange die Nutzer nichts an den Sucheinstellungen ändert.

/PL20/

Name: Effizienz der Suche

Akteure: Nutzer

Kurzbeschreibung: Nutzer darf nie mehr als eine halbe Sekunde auf seine Ergebnisse warten und die App soll in der Lage sein erweiterbare Datensätze ohne Leistungseinbrüche zu verwalten und darzustellen.

/PL30/

Name: Vollständigkeit und Funktionsfähigkeit der Nutzerfunktionen

Akteure: Nutzer und Admin

Kurzbeschreibung: Die Funktionen der Nutzerregistrierung und -verwaltung muss für alle gleichermaßen gewährleistet sein, es darf keine Einschränkungen geben, sodass nicht die komplette App genutzt werden kann. Der Admin bekommt im Gegensatz zum normalen Benutzer extra Funktionen, die entsprechend in dem Navigationsmenü als Reiter angezeigt werden.

/PL40/

Name: Richtigkeit der Verkehrsschilder

Akteure: Admin

Kurzbeschreibung: Die Verkehrsschilder müssen alle in ihren Eigenschaften der Realität entsprechen. Bei dem Hinzufügen neuer Verkehrsschilder muss mithilfe der Datenbank überprüft werden, dass es zu keinen Dopplungen kommt und Verkehrsschilder nur bestimmte Eigenschaften haben können. Außerdem müssen alle Eingaben richtig übernommen werden und dürfen keine Formatierungsfehler oder Ähnliches enthalten.

/PL50/

Name: Speicherkapazität

Kurzbeschreibung: Die App muss zulassen, alle gesetzlichen Verkehrsschilder, ohne Einbußen in der Leistung, speichern, verwalten und präzise ausgeben zu können.

/PL60/

Name: Kompatibilität

Kurzbeschreibung: Die App wird ausschließlich für Android entwickelt und sollte somit ohne Probleme auf den meisten Android-Versionen laufen.

/PL70/

Name: Intuitive und effiziente Verwendung

Akteure: Nutzer*in

Kurzbeschreibung: Die Verwendung der App, besonders die der Suchfunktionen sollte intuitiv und effizient geschehen. Die Nutzer sollen nicht zu lange damit verbringen die gewünschten Verkehrsschilder zu finden, sondern mit den bereitgestellten Filteroptionen schnell ihr gesuchtes Verkehrsschild finden.

4 Darstellung des Entwurfs der Software-Architektur

Die Software-Architektur wird nun anhand von den verwendeten Softwarepattern, Entwurfsprinzipien sowie den benötigten UML-Diagrammen dargestellt. Die Diagramme stellen sowohl die vorhandenen Komponente und Klassen, als auch deren Abhängigkeit und Zusammenarbeit dar.

4.1 Softwarepattern

Singleton:

Das Singleton-Pattern zeichnet sich dadurch aus, dass nur eine einzige Instanz einer Klasse existieren kann, die einen globalen Zugangspunkt hat. Das Pattern eignet sich besonders für Objekte, welche eine Datenbankverbindung herstellen. Problematisch an diesem Pattern ist, dass es schwer zu testen ist, da es mehrere Zugangspunkte hat.

4.2 Entwurfsprinzipien

4.2.1 DRY – *Don't Repeat Yourself*

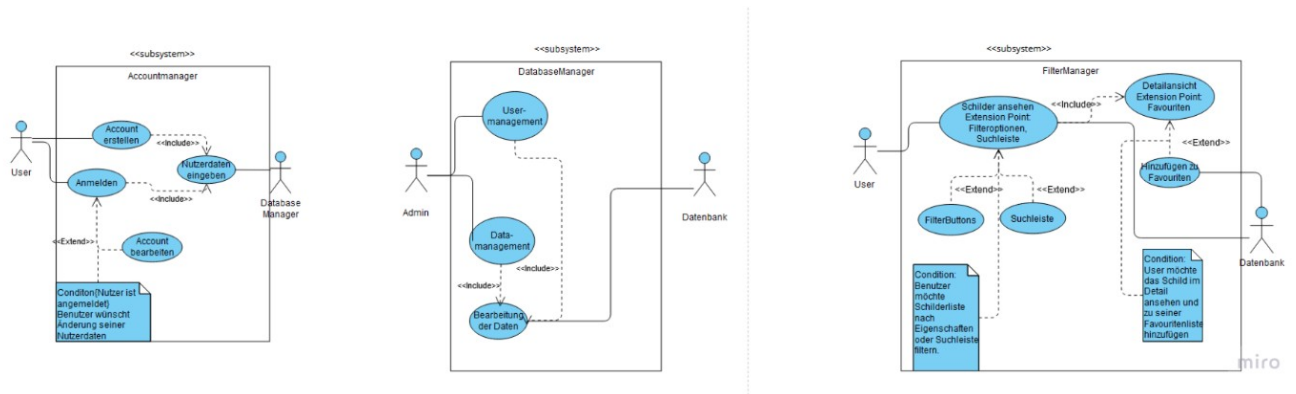
Das Entwurfsprinzip "DRY" wurde angewandt in der Hinsicht, dass spätestens ab der dritten Wiederholung einer Operation eine eigene Funktion dafür geschrieben wurde und somit vermieden wird, dass die Codebase undurchsichtig wird. Mit extra Funktionen die eigene Anforderung und Operationen erfüllen, lässt sich besser die Struktur und der Überblick über die Applikation halten.

4.2.2 Separation of Concerns

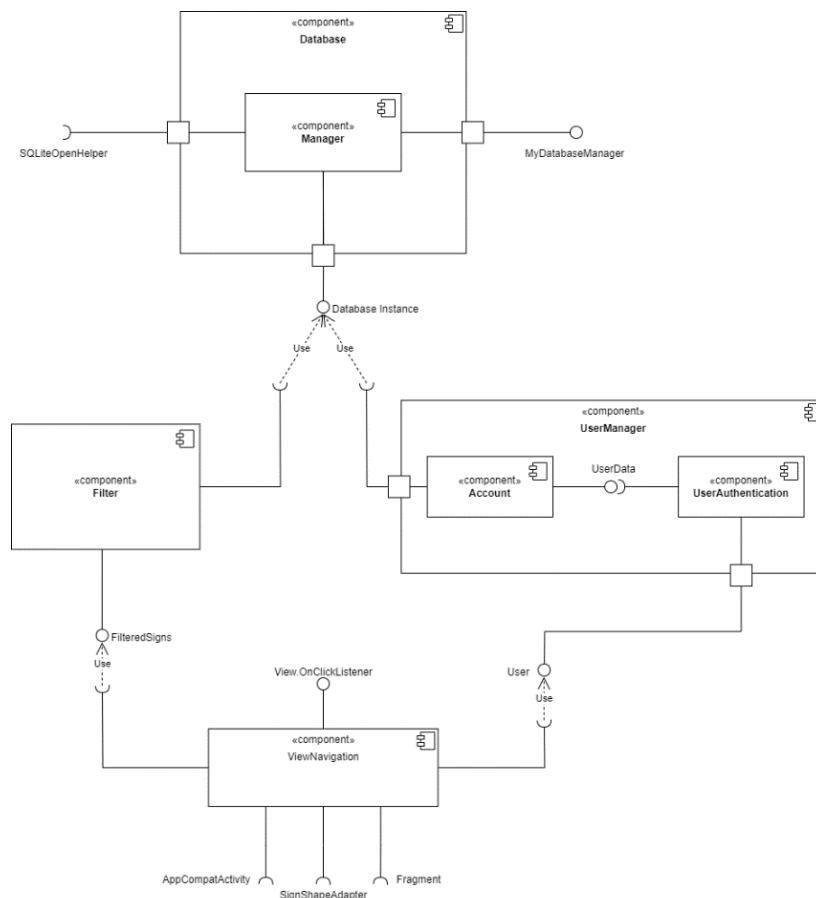
Durch die ansatzweise Umsetzung dieses Entwurfsprinzip wurde versucht Klassen, die zusammen eine übergeordnete Funktion erfüllen in ein eigenes Package zu verlegen, um die Strukturierung des Projektes zu garantieren, sollte es zukünftig größer skaliert werden.

4.2 UML-Diagramme

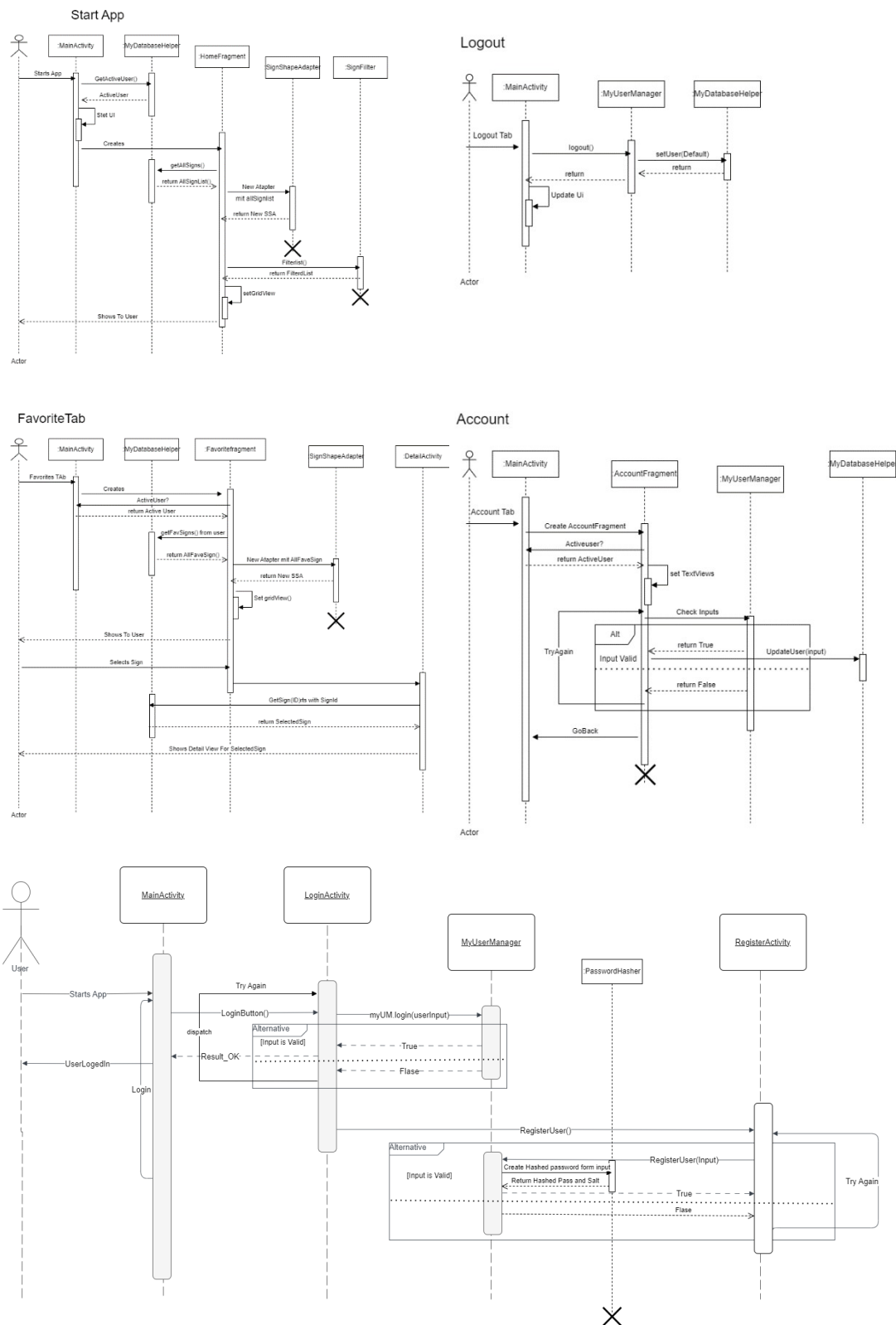
4.2.1 Use-Case-Diagramm:



4.2.2 Component Diagramm:

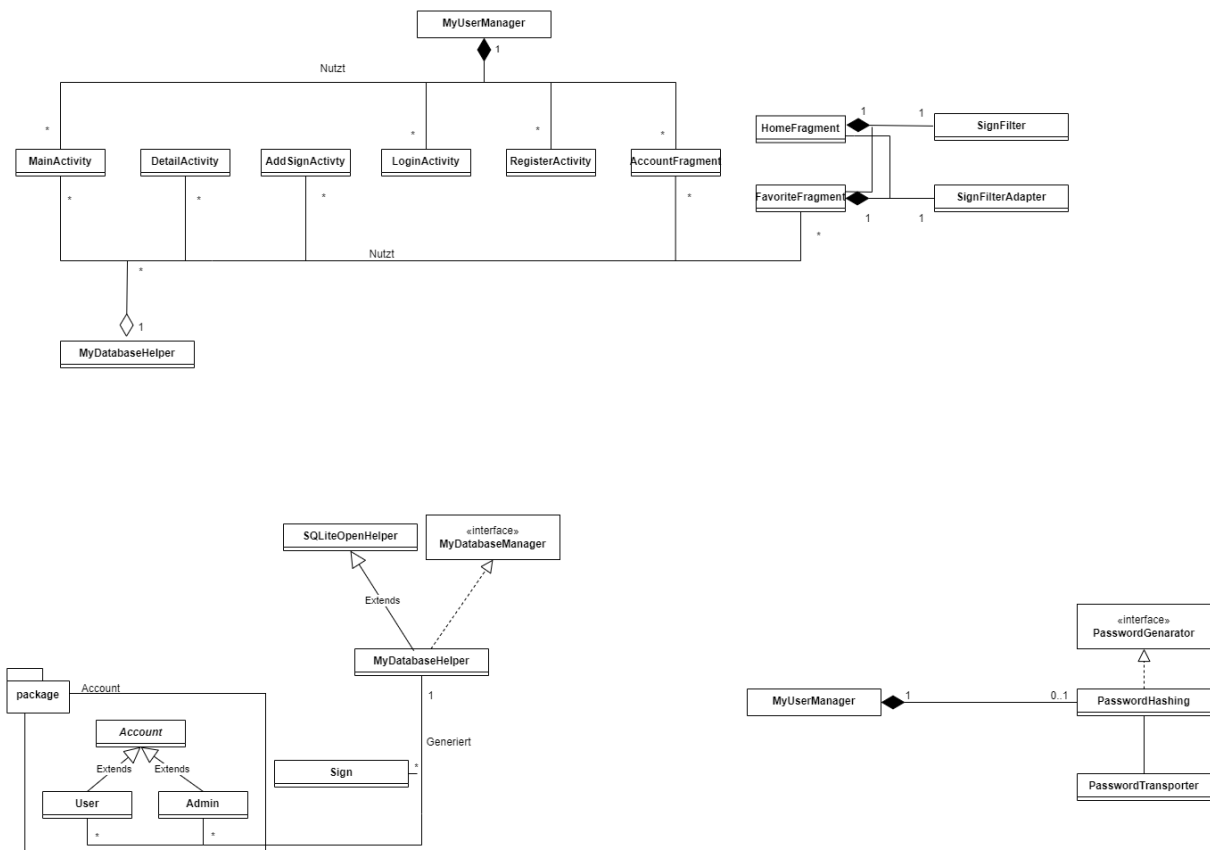


4.2.3 Sequenzdiagramm:



Oberes Sequenzdiagramm ist der Ablauf der App ohne das Login System
Unteres zeigt das Login/Registrierte System

4.2.4 Klassendiagramme



5 Darstellung der Implementation

Nachdem die benötigten Diagramme grob erstellt sowie im Lastenheft und Pflichtenheft, soweit möglich, alle wichtigen Funktionen, welche die App benötigt, festgehalten wurden, begann die Implementation. Dazu teilte sich die Gruppe in verschiedene Bereiche auf, wobei sich dies annähernd als „Frontend“ und „Backend“ darstellen lässt. Da die App allerdings lokal läuft und als „Eins“ fungiert, existiert keiner richtiger Frontend- und Backendteil, da keine Anbindung über beispielsweise API-Anfragen erfolgt. Jedoch erwies sich diese Aufteilung als vorteilhaft, da so die einzelnen Komponenten unabhängig voneinander aufgebaut und im Laufe des Projektes zusammengeführt wurden.

Die App besteht größtenteils aus vier Komponenten. Zunächst bildet die Datenbank die Grundlage, mit welcher die gesamten Daten gespeichert und auf dem lokalen Speicher des Gerätes abgelegt werden. Diese zu erstellen, zu erweitern und anzubinden führt zu einer guten Basis, auf der aufgebaut werden kann. Es wurden Funktionen geschrieben, um User, Verkehrsschilder und deren Eigenschaften sowie Bilder einzufügen, zu bearbeiten und zu löschen.

Die zweite wichtige Komponente besteht aus den Filtern, welche dafür sorgen, dass die Verkehrsschilder nach ihren Eigenschaften sortiert und angezeigt werden können. Dies zu implementieren, stellte ebenfalls eine essenzielle Aufgabe dar, da unter anderem auch die User Experience davon abhängt, wie schnell und einfach zu bedienen diese Filter sind. Daher war die Anforderung, welche am Anfang festgelegt wurde, dass die Suche nicht mehr als 0,5 Sekunden dauern darf. Dies wurde auch erreicht, wobei sich insgesamt nicht darauf konzentriert worden ist, die Performance auf das kleinste Detail hin zu optimieren. Dafür ist der Umfang der App schlicht zu klein ist und die Erfahrung in diesem Bereich, so wie Android-Studio nicht vorhanden.

Eine weitere Komponente stellt die Nutzerverwaltung dar, mit welcher Accounts dargestellt werden, welche einige Features mit sich bringen. Einerseits gibt es Adminaccounts, mit denen Verkehrsschildern editiert werden können, andererseits gibt es normale Nutzer, welche die Möglichkeit bekommen, Favoritenlisten zu erstellen um schnell an gewünschte Verkehrsschilder zu gelangen. Dieses Feature ist momentan nicht so umfangreich ausgeprägt, da die großen Vorteile einer solchen Nutzerverwaltung erst zum Vorschein kommen, wenn die App beispielsweise auf Cloud-Services oder eine Server-Client-Lösung expandiert wird, da dort Accounts auf andere Geräte übertragen werden können. Ein geplantes „Schnickschnack“-Feature beinhaltete die Implementation einer Trainingsfunktion pro Nutzer. Dies ist aus zeitlichen und Prioritätsgründen nicht zustande gekommen, kann aber erweitert werden.

Zum Abschluss fehlt noch die Komponente der Darstellung und UI. Diese wurde mit den in Android-Studio enthaltenen „Activities“ und „Fragmenten“ realisiert. Dort werden die Daten bei der Datenbank abgefragt und entsprechend in der App präsentiert, sodass Nutzer sich gut orientieren können und eine gute Experience erhalten. In einem GridView werden mit Hilfe von sogenannten „Drawables“ die Bilder der Verkehrsschilder dargestellt, sodass eine intuitive und schnelle Suche.

Die Komponenten wurden mit dem Hintergedanken implementiert, diese einfach austauschen zu können, indem sie zum Beispiel, in eigene Packages gelegt und somit die Verwaltung deutlich vereinfacht wird. Außerdem setzt diese Aufteilung voraus, dass lediglich Komponente beziehungsweise Klassen in demselben Package voneinander abhängen können, da sie ähnliche oder eine gemeinsame Funktion erfüllen.

Diese Komponente bilden den funktionierenden Prototypen der App und arbeiten soweit zusammen, dass es keine bekannten Bugs gibt, die trivial für die Funktion der App sind.

Abschließend gesagt ist zu erkennen, dass sich die Implementation als herausfordernd, aber nicht unmöglich gestaltete. Niemand aus der Gruppe hatte im Vorhinein Erfahrung mit Android-Studio, lediglich mit der Programmiersprache Java. Daher sind an einigen Stellen noch Verbesserungen möglich.

6 Test

Die Tests wurden nach Abschluss der Programmierung geschrieben. Insbesondere wurden Tests für die jeweiligen Hauptfunktionalitäten, MyDatabaseHelper, die Filterfunktion und den MyUserManager, geschrieben und durchgeführt. Aufgrund der Zeit- und Umfangsbeschränkungen des Projekts wurde es dabei belassen. Die Unit-Tests verifiziert die Spezifikation unserer Komponenten.

Danach wurde der Integrationstests durchgeführt, um zu überprüfen, ob alle Komponenten wie geplant zusammenarbeiten. Dies verlief im Großen und Ganzen wie erwartet. An ein paar Schnittstellen mussten jedoch die Übergänge angepasst werden, da sonst einige Bugs aufgetreten sind. Zum Beispiel wurde die Art wie die Schilder für die Detailansicht aus der Datenbank geholt werden, geändert.

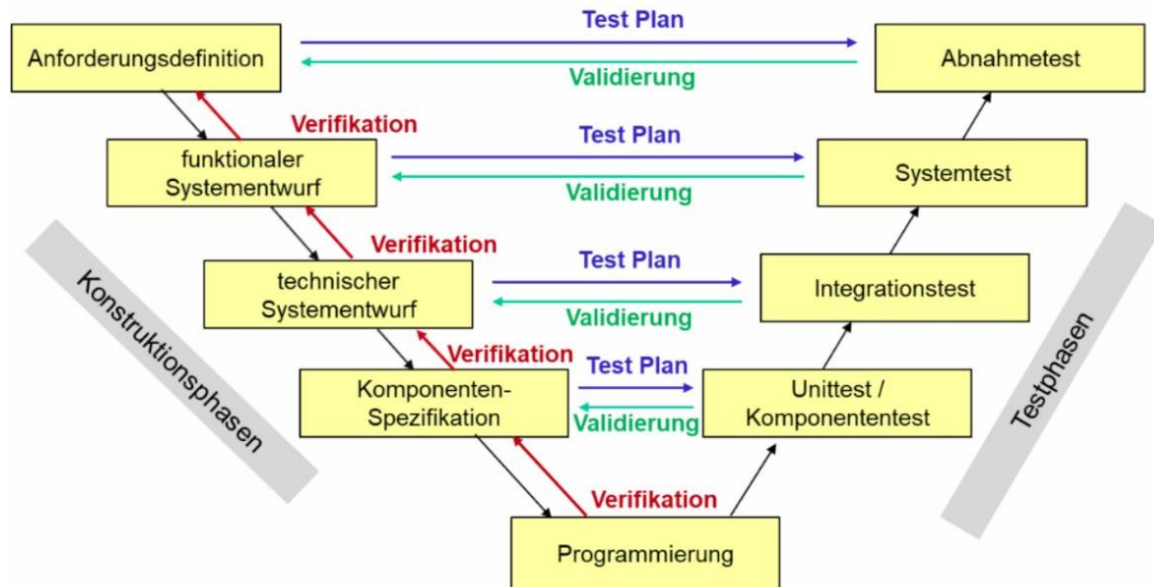
Anschließend lief der Systemtest wie geplant ab.

7 CI/CD

Aufgrund der Tatsache, dass unsere App vollständig lokal läuft, war es nicht möglich, eine umfassende CI/CD-Integration zu implementieren. Unabhängig davon beinhaltete das Ziel ebenfalls, die App so zu gestalten, dass sie einfach erweiterbar ist und Komponente sich ohne großen Aufwand austauschen lassen. Dies haben wir erreicht, indem wir beispielsweise Interfaces für die Datenbank und den Passwort-Generator erstellt haben. Auf diese Weise ist sofort klar, welche Methoden benötigt werden, wenn etwas ausgetauscht oder erneuert werden muss. Zudem haben wir versucht, die Inhalte weitgehend aus der Datenbank zu beziehen, sodass bei Änderungen nur die Datenbank angepasst werden muss.

8. Vorgehensmodell und Zusammenarbeit

8.1 Vorgehensmodell: Das V-Modell



8.2 Zusammenarbeit im Team

Wir haben unser Projekt gemäß dem V-Modell durchgeführt, indem wir gemeinsam die Anforderungsdefinition, den Systementwurf und die Komponentenspezifikation erarbeitet haben. Während der Programmierphase teilten wir die Rollen untereinander auf und folgten der schrittweisen Vorgehensweise des V-Modells. Die Tests wurden im Anschluss durchgeführt, genaueres findet sich im Testabsatz „Test“.

Name (Backend) konzentrierte sich hauptsächlich auf die Datenbank (Schema im Miro) und deren Implementierung mit Hilfe des Database Managers.

Name (Backend, Frontend) war für das Accountmanagement zuständig und unterstützte zusätzlich in anderen Bereichen, wo Hilfe benötigt wurde. Er erstellte unter anderem die Login/Register- sowie die Edit-Seite.

Name (Frontend, Backend) implementierte die Filter/Suche, den HomeView, DetailView und das Menü.

Ähnlich dem Scrum-Ansatz führten wir wöchentliche Meetings von jeweils einer Stunde durch. In diesen Besprechungen teilten wir die wesentlichen Aufgaben neu auf, diskutierten auftretende Probleme und erarbeiteten gemeinsam Lösungsvorschläge.

8.2.1 Aufwandseinschätzung:

~ 50-60 Stunden pro Person

Für genauere Information siehe Personal Log im Miro Board².

9 Lessons Learned

Tests während der Entwicklung schreiben:

Nachdem eine wichtige Funktionalität entwickelt wurde, hätten wir direkt den UnitTest dafür schreiben sollen. Das hätte uns sehr bei dem Voranschreiten der Programmierung geholfen, da man nach jeder kleinen Änderung am Code nicht nochmal die ganze Funktionalität der App per ausprobieren muss. Des Weiteren kann man Fehler so sehr schnell ausfindig machen, was sehr hilfreich ist, wenn man mit mehr als nur einer Person an derselben Funktionalität arbeitet.

Es wäre hilfreich, die gesamte UI, welche durch die gesamte App verteilt ist, auf eine Zentrale UI-Klasse zu bündeln um neben der besseren Übersichtlichkeit, auch das Ändern und Erweitern einfacher ist. (Stichwort: Separation of Concerns)

Mehr auf Code Lesbarkeit achten, um das Arbeiten mit den Teamkollegen zu vereinfachen.

Viele Erfahrungen sammeln, da es sonst sehr schwer ist die gesamte Planung, insbesondere die Softwarearchitektur vor auszuplanen.

10 Links

Miro Board:

<https://miro.com/welcomeonboard/...>

² siehe 10 Links